Questions on the TA code

From comparing `references/CodeByTA/` (`main_whole_data_PCA_batch.ipynb`, `Main.py`, `Digraph_generation_PC.R`, `Diagraph_generation_PC.R`) with our pipeline. Notebook lines are counted inside the cell (cell number in file order, cell id in brackets).

Data cleaning

Q1. Why are tags with any negative value dropped?

```
main_whole_data_PCA_batch.ipynb, cell 7 (df710e59), lines 86–94:
    negative_cols = df_num.columns[(df_num < 0).any()].tolist()
    df_num = df_num.drop(columns=negative_cols)
```

This removes 63 tags (cell 8 output), including pressure and differential-pressure tags that can be negative in normal operation (e.g. `02PDI215.PV`, `PDI3215A.PV`, `71PC1701.PV`). A single negative sample removes the whole tag. Is this meant to remove faulty sensors, or are negative readings invalid for these tags? We have not applied this rule yet.

T^2 threshold

Q2. Why is the T^2 limit multiplied by 1.2 (and 1.5) instead of using the base formula?

```
main_whole_data_PCA_batch.ipynb, cell 12 (3e429421), lines 58–64:
    T2_limit = (1.2 * (p*(n-1)*(n+1) / (n*(n-p))) * f.ppf(0.99, p, n-p))
cell 13 (834edfcf), line 58:  T2_limit = (1.5 * ...)
Main.py, lines 258–259:
    Tsq_thrs[P] = (((WL + P) * (WL - P)) / (WL * (WL - P))) * f.ppf(1 - alpha1, P, WL - P)
```

- How was the factor chosen (to reduce the number of alarms?), and which one should be used?
- The notebook uses the 0.99 quantile, while `Main.py` takes α from the user (line 46, e.g. 0.05). Which α is intended?
- The notebook formula has the factor p; `Main.py` lines 258–259 (and report Eq. 13) do not. Without p, the limit is about 1–2 while a normal T^2 averages about P. Which form is intended?

We currently use the base form $p(m+1)(m-1)/(m(m-p))\cdot F(1-\alpha; p, m-p)$ with $\alpha = 0.05$ and no multiplier.

PC algorithm

Q3. The test is the same (pcalg `gaussCItest` = Fisher-Z partial correlation, which we also use). Only the parameters differ:

```
Digraph_generation_PC.R, line 12: alpha1 <- 0.05 ; lines 27–29:
    pc(suffStat = list(C = cor(data), n = n), indepTest = gaussCItest,
       alpha = alpha1, fixedEdges = NULL, fixedGaps = FG, labels = V, verbose = FALSE)
lines 15, 23: FG from Fixed_gaps.xlsx   (Diagraph_generation_PC.R: lines 8, 19, 22–24)
Main.py, line 160: the same alpha1 is passed to PC and used for the T² limits
```

- **α:** you use 0.05 for PC and for T^2. We use 0.01 for PC (with this many rows Fisher-Z keeps very weak edges at 0.05). Is one α for both intended?
- **fixedGaps:** which edges does `Fixed_gaps.xlsx` forbid, and from what knowledge? Is there one for the 13-unit flare case? We forbid none.
- **Depth and rows:** pcalg runs with no limit on the conditioning-set size (`m.max = Inf`) on every row. For speed we cap it at 3 and use every 10th row (1-min data are strongly autocorrelated). Is that acceptable?
- **Orientation options:** you use pcalg defaults (`u2pd = "relaxed"`, `conservative = FALSE`, `maj.rule = FALSE`, `solve.confl = FALSE`)? We use causal-learn's `uc_rule = 0`, `uc_priority = 2`.

Graph orientation (for your information)

Q4. We orient the PC output automatically with Dor–Tarsi instead of stopping on a cycle. Is that acceptable?

```
Digraph_generation_PC.R, line 42: adj_matrix <- as(pc.fit@graph, "matrix")
Main.py, lines 194–197 and 364–365: any cycle -> "Digraph has cycle: Construct New Adjancy
matrix"
```

In `as(pc.fit@graph, "matrix")` an undirected edge appears in both directions, so `nx.simple_cycles` finds a 2-cycle and the run stops until the matrix is fixed by hand. We turn the PC output (a CPDAG) into a DAG with Dor & Tarsi (1992), the same algorithm as pcalg's `pdag2dag()`. It picks one DAG of the equivalence class without adding or removing edges. If no consistent DAG exists, we take the sink with the fewest violations, and reverse the fewest edges to break directed cycles. Both are logged and counted.

Overall T^2 (adopted) and a request

Q5. We now compute the overall T^2 as you do. Please share `function_file/` so we can check the decomposition.

```
Main.py, lines 229–231 and 243 (moving window: 302–304, 315):
    Cov_mat = np.corrcoef(Window_normal, rowvar=False)
    Tsq_sample = diag(Mat_normalized · pinv(Cov_mat) · Mat_normalizedᵀ)
Main.py, lines 9–15: tsq_decomp, NAN_remove, copula_transformed, tsq_stats (not included)
```

$T^2 = Z^T$ pinv(R) Z with the sample correlation R matches Eq. 7 of Li, Jin & Shi (2008), so we adopted it. With a sample R and a learned graph, the per-variable terms only add up to this T^2 when the graph is exactly right (their Proposition 1). Does `Tsq_decomp` (Main.py lines 246, 319) use each variable's parents from `adj_matrix` only, and does `Lcond` equal the number of parents?